JavaScript Object Iterations

Previous Next

General Methods

```
// Copies properties from a source object to a target object
Object.assign(target, source)

// Creates an object from an existing object
Object.create(object)

// Returns an array of the key/value pairs of an object
Object.entries(object)

// Creates an object from a list of keys/values
Object.fromEntries()

// Returns an array of the keys of an object
Object.keys(object)

// Returns an array of the property values of an object
Object.values(object)

// Groups object elements according to a function
Object.groupBy(object, callback)
```

JavaScript Object.assign()

The `Object.assign()` method copies properties from one or more source objects to a target object.

```
// Create Target Object
const person1 = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
};

// Create Source Object
const person2 = {firstName: "Anne",lastName: "Smith"};

// Assign Source to Target
Object.assign(person1, person2);
```

Try it Yourself »

JavaScript Object.entries()

ECMAScript 2017 added the `Object.entries()` method to objects.

`Object.entries()` returns an array of the key/value pairs in an object:

Example

```
const person = {
  firstName : "John",
  lastName : "Doe",
  age : 50,
  eyeColor : "blue"
};

let text = Object.entries(person);
```

Try it Yourself »

`Object.entries()` makes it simple to use objects in loops:

Example

```
  text += fruit + ": " + value + "<br>";
}
```

Try it Yourself »

`Object.entries()` also makes it simple to convert objects to maps:

Example

```
const fruits = {Bananas:300, Oranges:200, Apples:500};

const myMap = new Map(Object.entries(fruits));
```

Try it Yourself »

`Object.entries()` is supported in all modern browsers since March 2017:

JavaScript Object.fromEntries()

The `fromEntries()` method creates an object from a list of key/value pairs.

Example

```
const fruits = [
  ["apples", 300],
  ["pears", 900],
  ["bananas", 500]
];

const myObj = Object.fromEntries(fruits);
```

Try it Yourself »

ADVERTISEMENT

JavaScript Object.values()

`Object.values()` is similar to `Object.entries()`, but returns a single dimension array of the object values:

Example

```
const person = {
  firstName : "John",
  lastName : "Doe",
  age : 50,
  eyeColor : "blue"
};

let text = Object.values(person);
```

Try it Yourself »

`Object.values()` is supported in all modern browsers since March 2017:

JavaScript Object.groupBy()

ES2024 added the `Object.groupBy()` method to JavaScript.

The `Object.groupBy()` method groups elements of an object according to string values returned from a callback function.

The `Object.groupBy()` method does not change the original object.

Example

```
  {name:"bananas", quantity:300},
  {name:"oranges", quantity:200},
  {name:"kiwi", quantity:150}
];

// Callback function to Group Elements
function myCallback({ quantity }) {
  return quantity > 200 ? "ok" : "low";
}

// Group by Quantity
const result = Object.groupBy(fruits, myCallback);
```

Try it Yourself »

Browser Support

`Object.groupby()` is an ES2024 feature.

JavaScript 2024 is supported in new browsers since March 2024:

Chrome 117	Edge 117	Firefox 119	Safari 17.4	Opera 103
Sep 2023	Sep 2023	Oct 2023	Okt 2024	May 2023

Warning

ES2024 features are relatively new.

Older browsers may need an alternative code (Polyfill)

Object.groupBy() vs Map.groupBy()

The difference between Object.groupBy() and Map.groupBy() is:

Object.groupBy() groups elements into a JavaScript object.

JavaScript Object.keys()

The `Object.keys()` method returns an array with the keys of an object.

Example

```
// Create an Object
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
};

// Get the Keys
const keys = Object.keys(person);
```

Try it Yourself »

JavaScript for...in Loop

The JavaScript `for...in` statement loops through the properties of an object.

Syntax

```
for (let variable in object) {
  // code to be executed
}
```

The block of code inside of the `for...in` loop will be executed once for each property.

Looping through the properties of an object:

Example

```
const person = {
  fname:" John",
  lname:" Doe",
  age: 25
};
```

Try it Yourself »

Learn More:

JavaScript Object Definitions

JavaScript Object Properties

JavaScript Object Methods

JavaScript Object Display

JavaScript Object Constructors

JavaScript Object this

JavaScript Object Distructuring

JavaScript Object Prototypes

JavaScript Object Iterations

JavaScript Object Management

JavaScript Object Getters & Setters

JavaScript Object Protection

JavaScript Object Reference

ADVERTISEMENT

ADVERTISEMENT

ADVERTISEMENT

ADVERTISEMENT

PLUS
SPACES
GET CERTIFIED
FOR TEACHERS
FOR BUSINESS
CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
Angular Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples

Get Certified

HTML Certificate
CSS Certificate

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning. Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookie and privacy policy.

Copyright 1999-2025 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.